Instruction Pipeline

CSE Sem-4

Instruction pipeline: In this processor a task is divided into different subtasks and executes different subtasks at different phases of the pipeline. A stream of instructions can be executed by a pipeline in an overlapped manner.

Instruction Execution Phases:

1. **Fetch Stage** – Fetches instructions from memory.
2. **Decode Stage** – Decode the functions to be performed and identifies the resources needed. Resources include registers, buses, functional units
3. **Issue Stage** – Reserves resources, operands are read from registers/memory.
4. **Execute Stage** – Instructions are executed in one or several execute stages.
5. **Write-back** – This stage is used to write results into registers or memory.

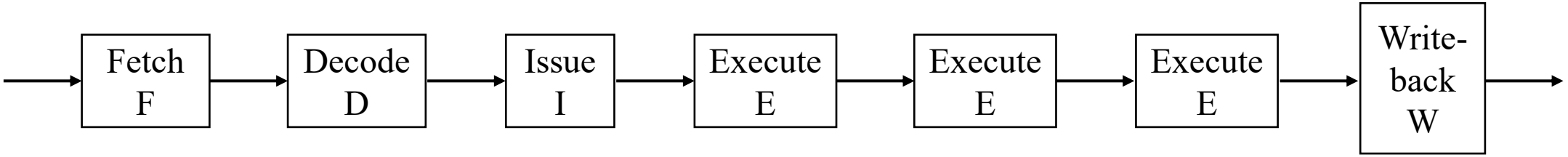
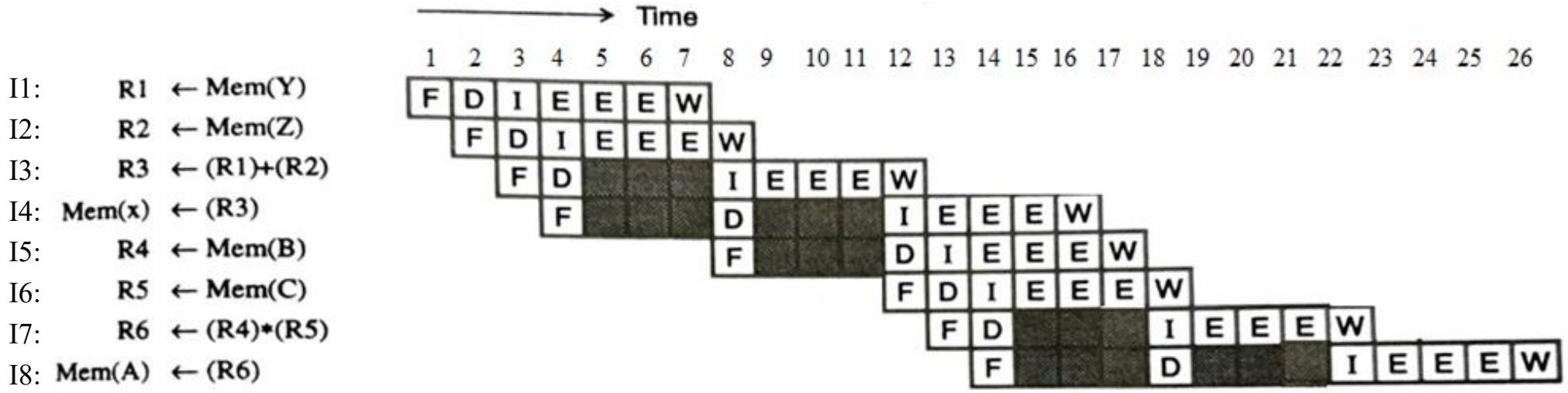
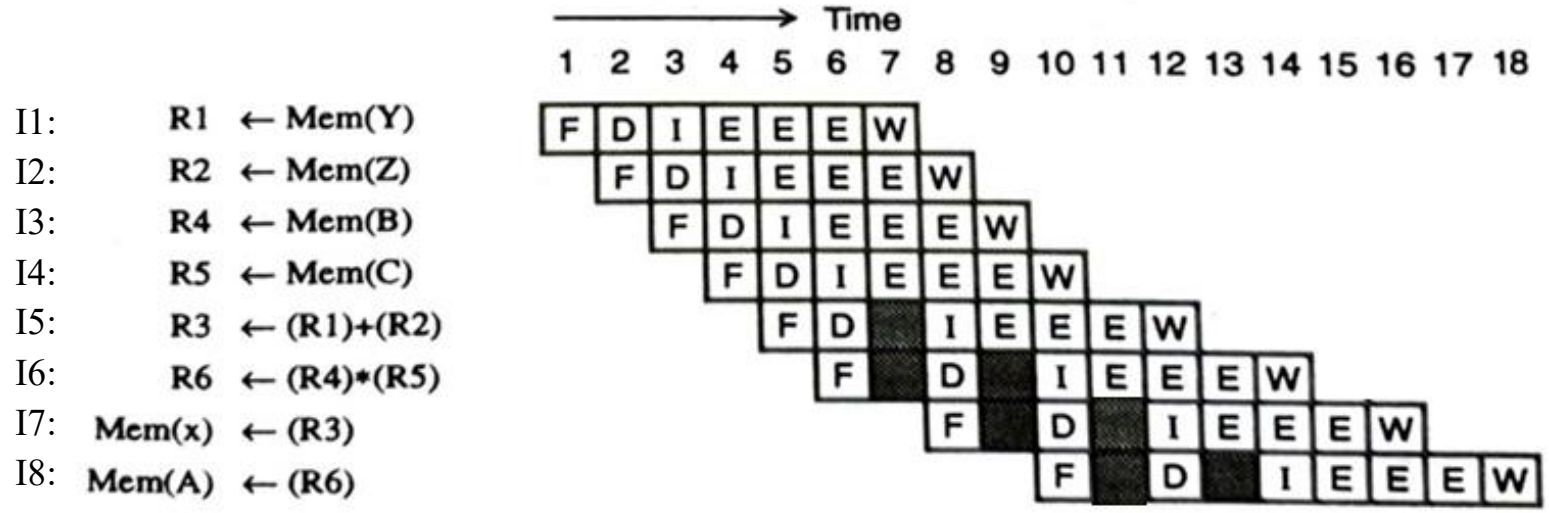


Fig: Seven stage pipelined architecture

❖ Pipelined Instruction Processing:



Stages	Operations
S1	Fetch
S2	Decode
S3	Issue
S4	Execute
S5	Execute
S6	Execute
S7	Write-back

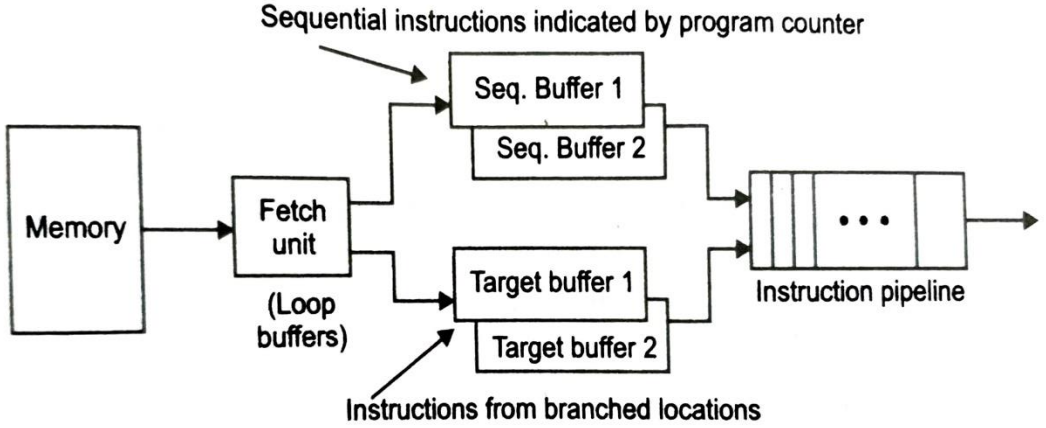


- Memory load or store operations are treated as part of the execution.

- **Pipeline Stall:** A condition when there will be a delay in the pipeline, i.e. pipeline stages will remain unutilized, then it is called pipeline stall.

❖ Mechanisms for Instruction Pipelining:

There are different mechanisms to smooth the pipeline flow.

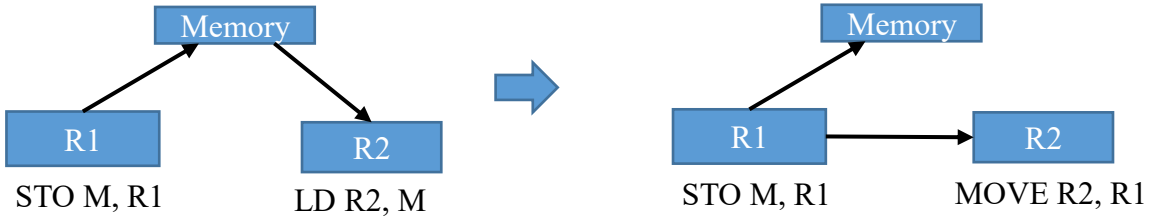


❑ **Prefetch Buffers:** In one memory access time, a block of consecutive instructions are fetched into a prefetched buffer.
It is of 3 types: **1) Sequential buffers** (for sequential instructions),
2) Target Buffers (for branching instructions),
3) Loop buffers (for instructions inside loop)

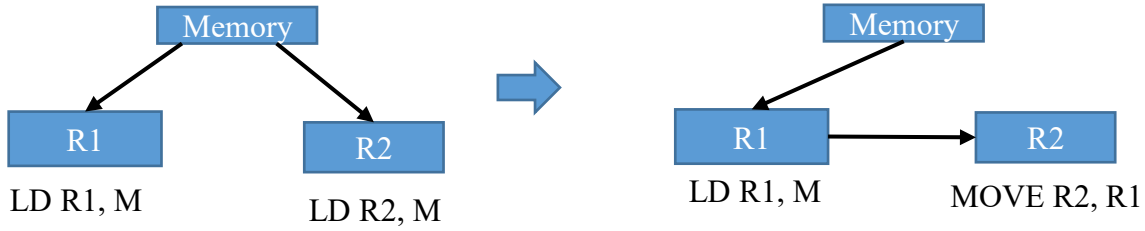
- **Sequential instructions** are loaded into the sequential buffers for in-sequence pipelining.
- **Branch instructions** are loaded into target buffers to perform out-of-sequence pipelining.
- A conditional branch instruction causes both sequential and target buffers to fill with instructions. After the branch condition is checked, appropriate instructions are taken from one of the two buffers, and instructions in the other buffer are discarded.
- Both buffers operates in **FIFO order**.
- **Loop buffer** holds sequential instructions contained in a small loop. Prefetched instructions will be executed in the loop body, until all instructions will complete its execution.
- Loop buffer operates in two steps: a) It contains those instructions that will be sequentially operated after the current instruction. It saves the fetch time. b) It recognizes when the target of a branch falls within the loop.
- The advantage of this scheme is that unnecessary memory access can be avoided.

- **Internal data forwarding**: Refers to a short circuit technique for replacing unnecessary memory accesses by register-to-register transfers in a sequence of fetch-arithmetic-store operations.

1. **Store-Load forwarding:** Suppose data from R1 will be stored in M and, same data from M will again be loaded into R2. So there are 2 memory access operations. These operations can be replaced by 2 parallel operations – one memory store and one register transfer operation. Since register transfer is faster than memory access, this data forwarding will reduce execution time.



2. **Load-Load forwarding:** It eliminates the 2nd level load operation (LD R2, M) and replaces with the move operation (Move R2, R1).



3. **Store-Store Overwriting:** Two stores from R1 and R2 to M are executed immediately, one after another, so that 2nd store overwrite the 1st store. Thus the 1st store becomes redundant and can be eliminated without affecting the result.



❑ ***Problem:*** *Consider the following operations in a sequence.*

1. R0 ← M1 (fetch)
2. R0 ← R0 + M2 (add)
3. R0 ← R0 × M3 (mul)
4. M4 ← R0 (store)

Apply internal-forwarding technique to simplify the sequences of such 4 equations, so that minimum number of memory references are done.

Solutions: 1) Fetching operand from M1 to R0.
2a) Transferring value of R0 to adder. 2b) Fetching operand from M2. 2c) Storing adder o/p to R0.
3a) Sending the adder o/p to multiplier. 3b) Fetching operand from M3. 3c) Storing multiplier o/p to R0.
4) Storing multiplier o/p to from R0 to M4.

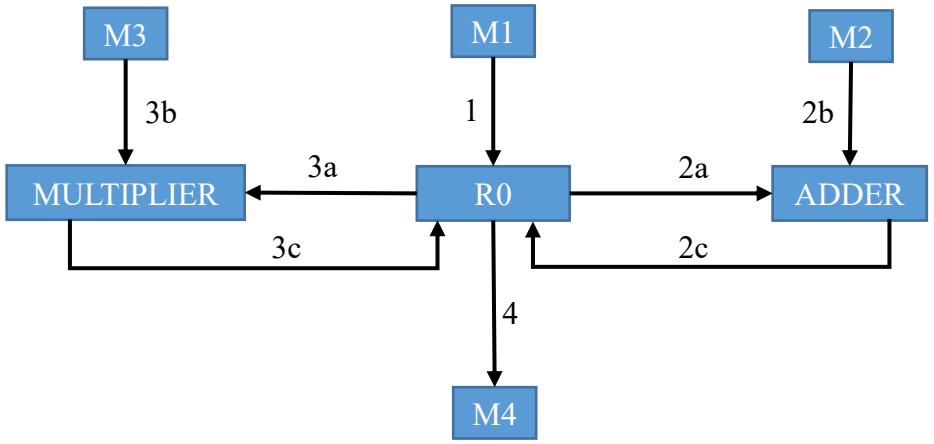


Fig 1: The original data flow sequence for the given problem

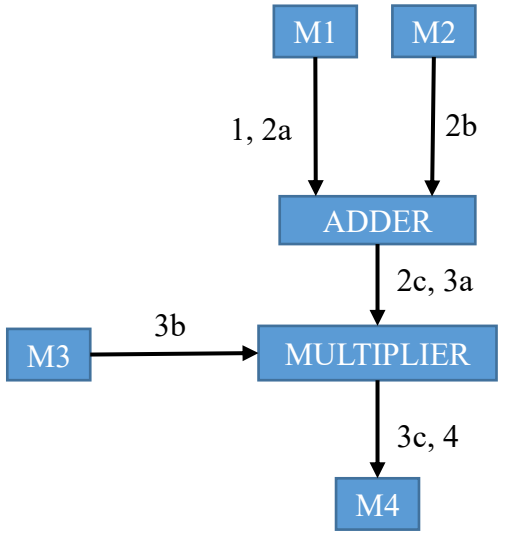


Fig 2: Data flow after inter-forwarding technique